

Contents

1	DD02: プロジェクト・FAQ 管理	1
1.1	0. 文書情報	1
1.2	1. 対象範囲	1
1.3	2. 収録ロジック・対応章	1
1.4	3. 詳細設計本文	2
1.4.1	3.1 画面詳細設計（参照のみ）	2
1.4.2	3.2 機能詳細設計（参照のみ）	2
1.4.3	3.3 データベース詳細設計（参照のみ）	2
1.4.4	3.4 実装モジュール構成	2
1.4.5	3.5 FAQ 公開ガード（AC-006）	2
1.4.6	3.6 FTS5 検索	2
1.4.7	3.7 埋め込みベクトル事前計算	3
1.4.8	3.8 一括インポート（faq.bulk_import）	3
1.4.9	3.9 リミット	3
1.4.10	3.10 プロジェクト作成時のオーナー自動 admin 付与（FR-015e / FR-030a）	3
1.4.11	3.11 プロジェクト削除時の論理削除カスケード + 孤立メンバー cleanup（FR-030b）	4
1.4.12	3.11a プロジェクト連絡先メール確認フロー（FR-033a / SCR-032）	4
1.4.13	3.12 90 日後物理削除バッチ（概要）	5
1.4.14	3.13 関連する横断設計	5
1.5	4. 関連設計	5
1.6	5. テスト観点	5
1.6.1	5.1 その他観点	5
1.7	6. 未確定事項・確認事項	6

1 DD02: プロジェクト・FAQ 管理

1.1 0. 文書情報

項目	内容
文書名	DD02: プロジェクト・FAQ 管理
詳細設計 ID	DD02
対象システム	FAQ AI ウィジェット SaaS / メインシステム
関連機能 ID	FR-040～048（FAQ CRUD / 公開ガード） / FR-100～103, FR-105, FR-106（未解決質問 → FAQ 登録）
作成日	2026-05-17
版数	v1.0
ステータス	承認済

1.2 1. 対象範囲

種別	ID	名称
機能	FR-040～048	FAQ CRUD / 自動公開禁止 / 公開・取下 / 一括インポート
機能	FR-100～103, FR-105, FR-106	未解決質問 → FAQ 登録（初期反映 / 確認・編集 / 登録元トレース）
画面	SCR-010	プロジェクト一覧
画面	SCR-011	FAQ 一覧
画面	SCR-012	FAQ 編集
API	/projects/*/faqs/*	プロジェクト・FAQ 操作
テーブル	projects faqs faq_embeddings faq_search_fts	FAQ 基盤

1.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 6 SCR-010～012	画面詳細設計	プロジェクト一覧 / FAQ 一覧 / FAQ 編集（正本は基本設計）
§ 7	機能詳細設計（FAQ 関連）	FAQ CRUD / 公開ガード / 一括インポート（正本は基本設計）
§ 9	データベース詳細設計	projects / faqs テーブル詳細（正本は基本設計）
§ 3.3.1	worker-main-api モジュール構成	routes/projects.ts routes/faqs.ts

1.4 3. 詳細設計本文

1.4.1 3.1 画面詳細設計（参照のみ）

SCR-010 プロジェクト一覧 / SCR-011 FAQ 一覧 / SCR-012 FAQ 編集の画面詳細は [../02_基本設計/01_画面設計.md](#) を正本とする。メッセージ ID は [../02_基本設計/06_メッセージ一覧.md](#) を参照。

1.4.2 3.2 機能詳細設計（参照のみ）

FAQ CRUD / 公開ガード / 一括インポートの全項目は [../02_基本設計/02_API 設計.md](#) を正本とする。

1.4.3 3.3 データベース詳細設計（参照のみ）

projects / faqs / faq_embeddings / faq_search_fts の DDL、CHECK 制約、インデックス、外部キー、コード値、SaaS データ分離観点（contract_owner_user_id）、保持期間は [../02_基本設計/03_テーブル設計.md](#) を正本とする。

1.4.4 3.4 実装モジュール構成

```
src/
├── routes/
│   ├── projects.ts      # /projects/* CRUD + メンバー割当
│   └── faqs.ts          # /faqs/* CRUD + 公開 / 取下 / 一括インポート / 提案承認
├── handlers/
├── domain/
│   └── faq-status.ts    # FAQ 状態遷移 (draft / review / published / unpublished)
├── repository/
│   ├── projects.ts
│   ├── faqs.ts
│   └── faq-embeddings.ts
├── middleware/
│   └── authorize.ts     # requireProject / requireRole 連動
```

1.4.5 3.5 FAQ 公開ガード（AC-006）

faqs.status は CHECK 制約で値域固定。status='published' への遷移は人手承認 API（POST /api/v1/faqs/{id}/publish）のみ許可。domain/faq-status.ts の canTransition() で draft → published を拒否、review → published のみ許可する。

1.4.6 3.6 FTS5 検索

faq_search_fts は SQLite の FTS5 virtual table で構築し、faqs.title || faqs.body をインデックス対象とする。FAQ 公開・更新時に同期更新。検索クエリは BM25 ランキングを利用：

```
SELECT f.*, bm25(faq_search_fts) AS rank
FROM faq_search_fts JOIN faqs f ON f.rowid = faq_search_fts.rowid
WHERE faq_search_fts MATCH ?1 AND f.project_id = ?2 AND f.status = 'published'
ORDER BY rank LIMIT 20
```

詳細は [DD03_AI 回答パイプライン.md](#) § 3.2 関連度計算を参照。

1.4.7 3.7 埋め込みベクトル事前計算

FAQ 公開時に @cf/baai/bge-base-en-v1.5 で埋め込みベクトルを計算し、faq_embeddings テーブル（または KV embedding:{faqId}）に保存。AI 推論時のリランキング用。

1.4.8 3.8 一括インポート (faq.bulk_import)

CSV/JSON ファイル → R2 ステージング → Queue 投入 → consumer が分割 INSERT。1 リクエスト最大 1000 件。詳細は [DD14_ バッチ・非同期処理.md](#) を参照。

1.4.9 3.9 リミット

項目	警告	拒否
FAQ 件数 / 契約	8,000	12,000 (409 FAQ_LIMIT_EXCEEDED)
プロジェクト数 / 契約	40	50 (409 PROJECT_LIMIT_EXCEEDED)

1.4.10 3.10 プロジェクト作成時のオーナー自動 admin 付与 (FR-015e / FR-030a)

POST /projects 実行時は SCR-010-M1(新規作成モード)から name/allowedDomains 等の基本情報のみを受け取る(initialAdmins フィールドは受け取らない)。オーナーは作成時に自動で当該プロジェクトの管理者となる(project_users(role='admin', valid=1) 自動 INSERT)。他者をプロジェクト管理者として招待する操作は、プロジェクト作成後に SCR-017-M1 経由(POST /projects/{id}/members+role='admin')で行う。

処理ステップ:

1. リクエスト検証: name(1~100 文字必須)、allowedDomains(1 件以上必須、形式チェック)。違反は 400 VALIDATION_ERROR
2. projects INSERT(ウィジェット公開 を発行、valid=1)
3. オーナー自動 admin 行 INSERT: INSERT INTO project_users(user_id=actor.contractOwnerUserId, project_id=newPid, contract_owner_user_id=actor.contractOwnerUserId, role='admin', valid=1, granted_at=now(), granted_by=actor.userId)
4. 監査ログ project.created_with_owner_admin(metadata に { projectId, contractOwnerUserId }、retention_class=general) を記録

擬似コード:

```
async function createProject(actor: Principal, req: CreateProjectRequest) {
  requireOwner(actor); // E-AUTHZ-OWNER-ONLY
  validateCreateProjectRequest(req); // 400
  return await db.transaction(async (tx) => {
    const project = await tx.insert('projects', {
      ...req,
      contract_owner_user_id: actor.contractOwnerUserId,
      valid: 1,
    });
    // オーナー admin 行を自動 INSERT(project_users)
    await tx.insert('project_users', {
      user_id: actor.contractOwnerUserId, // オーナー本人 = contract_owners.user_id = users.id
      project_id: project.id,
      contract_owner_user_id: actor.contractOwnerUserId, // 認可境界クエリ高速化の冗長保持
      role: 'admin',
      valid: 1,
      granted_by: actor.userId,
      granted_at: now(),
    });
    await tx.insertAudit({
      action: 'project.created_with_owner_admin',
      target_type: 'project',
      target_id: project.id,
      metadata: { contractOwnerUserId: actor.contractOwnerUserId },
      retention_class: 'general',
    });
    return project;
  });
}
```

```
});
}
```

1.4.11 3.11 プロジェクト削除時の論理削除カスケード + 孤立メンバー cleanup(FR-030b)

DELETE /projects/{id} 実行時は **SCR-026 のみ** から起動可能。以下のトランザクション境界で論理削除する:

1. 削除前のスナップショット: 当該プロジェクトに **valid=1** で割当のあるメンバー **userId** 一覧を取得 (オーナー含む)
2. UPDATE project_users SET valid=0, updated_at=now() WHERE project_id=? AND valid=1 (オーナー自身の admin 行も含む全行を論理削除)
3. 孤立メンバーの抽出: 「ステップ1のメンバーのうち、他プロジェクト割当 (**valid=1** の project_users) が0件かつ **contract_owners** 行を持たない (非オーナー) もの」
4. 孤立メンバーの全セッションを失効 (UPDATE sessions SET revoked_at=now()) + 未使用招待トークンを失効 (UPDATE access_tokens SET used_at=now() WHERE used_at IS NULL) + UPDATE users SET valid=0, updated_at=now() で論理削除
5. UPDATE projects SET status='deleted', valid=0, deleted_at=now(), updated_at=now() WHERE id=? + 関連テーブル (allowed_domains / faqs / inquiries / end_users / chat_rooms / question_logs) を valid=0, updated_at=now() に伝播
6. 監査ログ project.logical_delete(metadata に { projectId, logicallyDeletedUserIds: [...]}、retention_class=general) を記録

オーナーの **users** 行は本処理の論理削除対象外 (**contract_owners** 行を持つユーザーは常に **valid=1** 維持)。当該プロジェクトに対するオーナー自身の **project_users** admin 行は他のメンバー行と同様に **valid=0** に論理削除される。論理削除データは **updated_at < now() - 90d AND valid=0** の物理削除バッチ (DD14_バッチ・非同期処理.md § 3.12) で 90 日後に物理削除される。

1.4.12 3.11a プロジェクト連絡先メール確認フロー (FR-033a / SCR-032)

SCR-010-M1 で **projects.contact_email** が新規入力 / 変更された時に、TPL-PROJECT_CONTACT_VERIFY 確認メールを連絡先メール宛に送信し、**access_tokens.purpose='contact_verify'** (24h、meta={"projectId": "..."}) を発行する。受信者は SCR-032 メンバーアカウント連絡先確認完了ページに着地し、POST /auth/contact-verifications/{token} 経由で確認完了。

// (1) 連絡先メール変更時 (POST/PATCH /projects 内)

```
await env.DB.prepare(
  `UPDATE projects SET contact_email = ?1, contact_email_verified_at = NULL, updated_at = ?2 WHERE id = ?3`
).bind(newContactEmail, now, projectId).run();
const rawToken = await generateToken('contact_verify', { projectId }, env, 24 * 3600);
await enqueueEmail({
  template: 'TPL-PROJECT_CONTACT_VERIFY',
  to: newContactEmail,
  verifyUrl: `${env.WEB_BASE}/auth/contact-verify?token=${encodeURIComponent(rawToken)}`,
});
```

// (2) SCR-032 着地時 (POST /auth/contact-verifications/{token})

```
const { payload } = await verifyToken(rawToken, 'contact_verify', env); // 410 if expired/used
const { projectId } = payload;
await env.DB.batch([
  env.DB.prepare(`UPDATE projects SET contact_email_verified_at = ?1, updated_at = ?1 WHERE id = ?2`),
]);
await consumeToken(rawToken, 'contact_verify', env);
await writeAudit({ action: 'project.contact_email_verified', target_project_id: projectId });
```

注意:

- 連絡先メール再変更時は旧トークンを **consumeToken()** で失効させ、新トークンを発行する
- **contact_email_verified_at** が NULL の間はウィジェット表示に用いない (FR-033c)
- 受信者 (連絡先メール所有者) はアカウントを持つ必要がない —— 第三者の共有メールアドレスでも本フローを進められる
- Turnstile は不要 (状態変更は単純な **verified_at** セットのみ、トークン HMAC + レート制限で十分)
- 詳細なトークン発行ロジックは DD08_トークン発行・検証.md § 3.3b を参照

1.4.13 3.12 90 日後物理削除バッチ (概要)

詳細は [DD14_ バッチ・非同期処理.md § 3.12](#) を正本とする。日次バッチで `valid=0 AND updated_at < now() - 90d` の行を物理削除する。`users` 行が物理 DELETE されると `ON DELETE CASCADE` で関連 `contract_owners` / `project_users` / `sessions` / `access_tokens` / `terms_agreements` 等が連鎖物理削除される。`projects` 行が物理 DELETE されると `ON DELETE CASCADE` で関連 `faqs` / `inquiries` / `chat_*` 等が連鎖物理削除される。`billing_subscriptions` は物理削除バッチの対象外 (電子帳簿保存法 7 年保持、永久維持)。匿名化モード (`contract_owners.data_deletion_mode='anonymize'`) は物理削除前に匿名化処理を実施する。

1.4.14 3.13 関連する横断設計

- 認可: [DD09_ 認可ヘルパ.md](#) の `requireProjectRole` でオーナー境界 + プロジェクトロール (`admin` / `member`) を検証
- 監 査 ロ グ: `faq.create` / `faq.update` / `faq.publish` / `faq.unpublish` / `faq.bulk_import` / `project.created_with_owner_admin` / `project.logical_delete` / `project.member_invited` / `project.hard_delete_by_batch` (90 日後物理削除バッチ起動) を `retention_class=general` で記録
- リアルタイム反映: FAQ 公開・編集時に `widget-api` 側の KV キャッシュ (`embedding:{faqId}`、`ai_threshold:{owner}:{project}` は別系統) を `invalidate`
- 論理削除フィルタ: 全 GET 系クエリで `WHERE valid=1` を必須付与 (漏れは IDOR 類似の脆弱性、09_セキュリティ設計参照)

1.5 4. 関連設計

種別	参照先
要件	../01_ 要件定義/index.md
基本設計	../02_ 基本設計/index.md
画面設計	../02_ 基本設計/01_ 画面設計.md
テーブル設計	../02_ 基本設計/03_ テーブル設計.md
API 設計	../02_ 基本設計/02_API 設計.md
運用設計	../04_ 運用設計/index.md
将来対応	../05_future/index.md
関連 DD	DD03_AI 回答パイプライン.md / DD09_ 認可ヘルパ.md / DD10_ 監査ログ書込・完全性検証.md / DD13_ ウィジェット配信.md

1.6 5. テスト観点

AC ID	テスト ID	テスト方式	テストファイル
AC-006	u-faq-publish-001 / it-faq-publish-001	Unit + Integration	workers/main-api/test/{unit,integration}/faq/publish-guard.test.ts
AC-014	e2e-faq-crud-001	E2E	apps/admin/e2e/faq/crud.spec.ts

1.6.1 5.1 その他観点

観点	内容
単体	faq-status.ts 遷移ガード全パターン
結合	FTS5 検索の MATCH 構文 / BM25 ランキング
異常系	他オーナーのプロジェクトに対する FAQ CRUD アクセス時 404
境界値	FAQ 件数 7999 / 8000 / 8001 / 11999 / 12000 / 12001
権限	該当プロジェクトに割当のないメンバー (<code>project_users</code> 行なし) による FAQ 編集試行で 404 偽装 / 該当 PJ の member+ ロールを持つメンバーは編集可
性能	POST /api/v1/faqs/{id}/publish p95 < 300ms (UPDATE 1 行 + KV invalidate)

1.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済